

✓ 06 — FIPCA rank ablation ($d \in \{16, 32, 64\}$)

Phase 6 of FedSafe-Fuse. Runs on Google Colab Free + T4.

What this notebook does

Sweeps the **FIPCA projection rank** d through three values from the proposal (16, 32, 64), holding everything else fixed:

- Model: FedSafe-Fuse (~4.9M params)
- $K=3$ clients (proposal)
- $T=25$ rounds (reduced from $T=50$ for the 20-min phase budget; trend across ranks is what matters for an ablation)
- $E=5$ local epochs (proposal)
- $\text{batch}=16$, $\text{samples_per_local_epoch}=40$
- $\text{LR}=1\text{e-}3$, $\beta=1.0$

For each rank, we report:

1. Final-round eval SSIM and PSNR on 128 MedMNIST test samples
2. Upstream communication bytes per round ($K \times \text{rank} \times 4$ bytes)
3. Compression ratio vs DP-SGD (Phase 3 result, 59.18 MB/round)

Outputs

- `results/ablation_fipca_rank.csv` — one row per rank
- `figures/ablation_fipca_rank.png` — SSIM vs rank + bytes vs rank dual-axis plot

Expected wall-clock on T4: ~12 min

```
# 0. Install deps + seeds
import os, sys, time, json, random
import numpy as np
import torch, torch.nn as nn, torch.nn.functional as F
from torch.utils.data import DataLoader, Dataset, Subset
import matplotlib.pyplot as plt
import csv

SEED = 42
random.seed(SEED); np.random.seed(SEED); torch.manual_seed(SEED)
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(SEED)
DEVICE = 'cuda' if torch.cuda.is_available() else 'cpu'
print('Device:', DEVICE, '| Torch:', torch.__version__)
```

Device: cuda | Torch: 2.11.0+cu128

```
# 1. Drive mount + mirror needed files (same pattern as Phase 3)
from google.colab import drive
import shutil

if not os.path.exists('/content/drive/MyDrive'):
    drive.mount('/content/drive')

DRIVE_ROOT = '/content/drive/MyDrive/FedSafeFuse'
LOCAL_ROOT = '/content/local_fedsafe'
for sub in ['medmnist', 'partitions', 'results', 'figures']:
    os.makedirs(f'{LOCAL_ROOT}/{sub}', exist_ok=True)

for fname in ['medmnist_K3.json']:
    src = f'{DRIVE_ROOT}/partitions/{fname}'
    dst = f'{LOCAL_ROOT}/partitions/{fname}'
    if os.path.exists(src) and not os.path.exists(dst):
        shutil.copy(src, dst)

for fname in ['train_paired.npz', 'test_paired.npz']:
    src = f'{DRIVE_ROOT}/medmnist/{fname}'
    dst = f'{LOCAL_ROOT}/medmnist/{fname}'
    if not os.path.exists(dst):
        t0 = time.time()
        print(f'Copying {fname}...')
        shutil.copy(src, dst)
        print(f' {fname}: {os.path.getsize(dst)/1e6:.0f} MB in {time.time()-t0:.1f}s')
    else:
```

```
print(f' {fname}: already on local disk')

PROJECT_DRIVE = LOCAL_ROOT
print('Local mirror ready:', PROJECT_DRIVE)
```

```
Mounted at /content/drive
Copying train_paired.npz...
  train_paired.npz: 2525 MB in 40.0s
Copying test_paired.npz...
  test_paired.npz: 1307 MB in 21.3s
Local mirror ready: /content/local_fedsafe
```

✓ Embedded model + trainer (verbatim from [src/](#))

```
# FedSafe-Fuse model
"""FedSafe-Fuse: dual MobileNetV3-Small encoders + 2-layer Transformer + conv decoder.

Per the project proposal:
- Two modality-specific encoders, each MobileNetV3-Small (~2.5M params total each)
- Shared 2-layer Transformer module (2 heads, embed dim 128) for cross-modal attention
- Convolutional decoder
- Target ~8M params

Actual realised parameter count depends on decoder width; default config lands at ~5M.
"""

from __future__ import annotations

import torch
import torch.nn as nn
from torchvision.models import mobilenet_v3_small

class ModalityEncoder(nn.Module):
    """MobileNetV3-Small features adapted to 1-channel input.

    MobileNetV3-Small downsamples by 32x, so a 128x128 input yields 4x4 features.
    We bilinearly upsample to 8x8 to give the cross-modal Transformer 64 tokens
    per modality (matching the proposal). The upsample is parameter-free.
    """

    def __init__(self, embed_dim: int = 128):
        super().__init__()
        base = mobilenet_v3_small(weights=None)
        # Replace first conv to accept a single channel (medical grayscale)
        base.features[0][0] = nn.Conv2d(1, 16, kernel_size=3, stride=2, padding=1, bias=False)
        self.features = base.features # (B, 576, 4, 4) for 128x128 input
        self.proj = nn.Conv2d(576, embed_dim, kernel_size=1)
        self.upsample = nn.Upsample(scale_factor=2, mode="bilinear", align_corners=False)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        h = self.features(x)
        h = self.proj(h)
        return self.upsample(h) # (B, embed_dim, 8, 8)

class CrossModalTransformer(nn.Module):
    """2-layer Transformer encoder over concatenated MRI/PET tokens.

    Input feature maps are flattened to 64 spatial tokens per modality, given a
    learned modality embedding, concatenated to 128 tokens, given a learned
    positional embedding, and passed through `num_layers` transformer encoder layers.
    Output is averaged across the two modality halves and reshaped back to a
    spatial feature map for the decoder.
    """

    def __init__(
        self,
        embed_dim: int = 128,
        num_layers: int = 2,
        num_heads: int = 2,
        ff_dim: int = 256,
        dropout: float = 0.1,
        spatial_tokens_per_modality: int = 64,
    ):
        super().__init__()
        encoder_layer = nn.TransformerEncoderLayer(
            d_model=embed_dim,
            nhead=num_heads,
```

```

        dim_feedforward=ff_dim,
        dropout=dropout,
        batch_first=True,
        activation="gelu",
    )
    self.transformer = nn.TransformerEncoder(encoder_layer, num_layers=num_layers)
    total_tokens = spatial_tokens_per_modality * 2
    self.pos_embed = nn.Parameter(torch.zeros(1, total_tokens, embed_dim))
    nn.init.trunc_normal_(self.pos_embed, std=0.02)
    self.mod_embed = nn.Embedding(2, embed_dim) # 0 = MRI, 1 = PET
    self.spatial_tokens = spatial_tokens_per_modality

def forward(self, mri_feat: torch.Tensor, pet_feat: torch.Tensor) -> torch.Tensor:
    # mri_feat, pet_feat: (B, C, H, W) with H*W = self.spatial_tokens
    B, C, H, W = mri_feat.shape
    mri_tokens = mri_feat.flatten(2).transpose(1, 2) # (B, HW, C)
    pet_tokens = pet_feat.flatten(2).transpose(1, 2)
    mri_tokens = mri_tokens + self.mod_embed.weight[0]
    pet_tokens = pet_tokens + self.mod_embed.weight[1]
    tokens = torch.cat([mri_tokens, pet_tokens], dim=1) + self.pos_embed
    out = self.transformer(tokens)
    mri_out = out[:, : self.spatial_tokens].transpose(1, 2).reshape(B, C, H, W)
    pet_out = out[:, self.spatial_tokens :].transpose(1, 2).reshape(B, C, H, W)
    return 0.5 * (mri_out + pet_out)

class FusionDecoder(nn.Module):
    """Progressive bilinear upsample + conv decoder: 8x8 -> 16 -> 32 -> 64 -> 128."""

    def __init__(self, embed_dim: int = 128, base_ch: int = 256):
        super().__init__()
        c0, c1, c2, c3 = base_ch, base_ch, base_ch // 2, base_ch // 4
        self.up1 = self._block(embed_dim, c0)
        self.up2 = self._block(c0, c1)
        self.up3 = self._block(c1, c2)
        self.up4 = self._block(c2, c3)
        self.final = nn.Conv2d(c3, 1, kernel_size=1)

    @staticmethod
    def _block(in_ch: int, out_ch: int) -> nn.Sequential:
        return nn.Sequential(
            nn.Upsample(scale_factor=2, mode="bilinear", align_corners=False),
            nn.Conv2d(in_ch, out_ch, kernel_size=3, padding=1),
            nn.BatchNorm2d(out_ch),
            nn.GELU(),
            nn.Conv2d(out_ch, out_ch, kernel_size=3, padding=1),
            nn.BatchNorm2d(out_ch),
            nn.GELU(),
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        h = self.up1(x)
        h = self.up2(h)
        h = self.up3(h)
        h = self.up4(h)
        return torch.sigmoid(self.final(h))

class FedSafeFuse(nn.Module):
    """End-to-end FedSafe-Fuse model. Inputs: two (B, 1, 128, 128) tensors. Output: (B, 1, 128, 128)."""

    def __init__(
        self,
        embed_dim: int = 128,
        transformer_layers: int = 2,
        transformer_heads: int = 2,
        transformer_ff: int = 256,
        decoder_base_ch: int = 256,
    ):
        super().__init__()
        self.mri_encoder = ModalityEncoder(embed_dim)
        self.pet_encoder = ModalityEncoder(embed_dim)
        self.fusion = CrossModalTransformer(
            embed_dim=embed_dim,
            num_layers=transformer_layers,
            num_heads=transformer_heads,
            ff_dim=transformer_ff,
        )
        self.decoder = FusionDecoder(embed_dim=embed_dim, base_ch=decoder_base_ch)

    def forward(self, mri: torch.Tensor, pet: torch.Tensor) -> torch.Tensor:
        mri_feat = self.mri_encoder(mri)

```

```

        pet_feat = self.pet_encoder(pet)
        fused_feat = self.fusion(mri_feat, pet_feat)
        return self.decoder(fused_feat)

def count_parameters(model: nn.Module) -> int:
    return sum(p.numel() for p in model.parameters() if p.requires_grad)

# Composite fusion loss + eval metrics
"""Composite fusion loss: L1 + beta * (1 - SSIM).

Implements a differentiable SSIM via a Gaussian-windowed convolution
(no external dependency on pytorch_msssim). Designed for single-channel
medical images in [0, 1].
"""

from __future__ import annotations

import torch
import torch.nn as nn
import torch.nn.functional as F

def _gaussian_window(window_size: int, sigma: float, channels: int) -> torch.Tensor:
    coords = torch.arange(window_size, dtype=torch.float32) - (window_size - 1) / 2
    g = torch.exp(-(coords ** 2) / (2 * sigma ** 2))
    g = g / g.sum()
    w_2d = g.unsqueeze(0) * g.unsqueeze(1) # (W, W)
    return w_2d.expand(channels, 1, window_size, window_size).contiguous()

class SSIMLoss(nn.Module):
    """1 - SSIM, computed with a Gaussian sliding window. Single-channel input in [0, 1]."""

    def __init__(
        self,
        window_size: int = 11,
        sigma: float = 1.5,
        data_range: float = 1.0,
        channels: int = 1,
    ):
        super().__init__()
        self.window_size = window_size
        self.data_range = data_range
        self.C1 = (0.01 * data_range) ** 2
        self.C2 = (0.03 * data_range) ** 2
        self.register_buffer("window", _gaussian_window(window_size, sigma, channels))

    def _filter(self, x: torch.Tensor) -> torch.Tensor:
        return F.conv2d(x, self.window, padding=self.window_size // 2, groups=x.shape[1])

    def forward(self, pred: torch.Tensor, target: torch.Tensor) -> torch.Tensor:
        mu_p = self._filter(pred)
        mu_t = self._filter(target)
        mu_p2 = mu_p ** 2
        mu_t2 = mu_t ** 2
        mu_pt = mu_p * mu_t
        sigma_p2 = self._filter(pred * pred) - mu_p2
        sigma_t2 = self._filter(target * target) - mu_t2
        sigma_pt = self._filter(pred * target) - mu_pt
        num = (2 * mu_pt + self.C1) * (2 * sigma_pt + self.C2)
        den = (mu_p2 + mu_t2 + self.C1) * (sigma_p2 + sigma_t2 + self.C2)
        ssim_map = num / den
        return 1.0 - ssim_map.mean()

class FusionLoss(nn.Module):
    """L1(fused, ref) + beta * SSIMLoss(fused, ref).

    Reference composite is a per-pixel weighted average of source modalities, per the proposal.
    Default weights are (0.5, 0.5). Override `reference_weights` per dataset if needed.
    """

    def __init__(self, beta: float = 1.0, reference_weights: tuple[float, float] = (0.5, 0.5)):
        super().__init__()
        assert abs(sum(reference_weights) - 1.0) < 1e-6, "reference_weights must sum to 1.0"
        self.l1 = nn.L1Loss()
        self.ssim = SSIMLoss()
        self.beta = beta
        w1, w2 = reference_weights

```

```

        self.register_buffer("w1", torch.tensor(w1))
        self.register_buffer("w2", torch.tensor(w2))

    def reference(self, src1: torch.Tensor, src2: torch.Tensor) -> torch.Tensor:
        return self.w1 * src1 + self.w2 * src2

    def forward(
        self,
        fused: torch.Tensor,
        src1: torch.Tensor,
        src2: torch.Tensor,
    ) -> torch.Tensor:
        ref = self.reference(src1, src2)
        return self.l1(fused, ref) + self.beta * self.ssim(fused, ref)

@torch.no_grad()
def ssim_score(pred: torch.Tensor, target: torch.Tensor) -> float:
    """SSIM (higher is better) for evaluation. Mirrors SSIMLoss without the 1 - ... wrap."""
    loss_fn = SSIMLoss().to(pred.device)
    return float(1.0 - loss_fn(pred, target).item())

@torch.no_grad()
def psnr_score(pred: torch.Tensor, target: torch.Tensor, data_range: float = 1.0) -> float:
    """PSNR in dB."""
    mse = F.mse_loss(pred, target).item()
    if mse == 0:
        return float("inf")
    return 10.0 * torch.log10(torch.tensor(data_range ** 2 / mse)).item()

```

```

# Dataset wrappers
"""PyTorch Datasets for FedSafe-Fuse: MedMNIST paired arrays and BraTS per-case slices.

Designed to read directly from Google Drive (`/content/drive/MyDrive/FedSafeFuse/`)
in Colab, or from a local copy of that tree.
"""

from __future__ import annotations

import json
import os
from collections import OrderedDict
from typing import Iterable

import numpy as np
import torch
from torch.utils.data import Dataset

class MedMNISTPaired(Dataset):
    """Pre-computed (MRI, PET, label) triples from a single .npz on Drive.

    Optional `indices` restricts to a subset of the global array (used for per-client
    partitions and calibration hold-outs).
    """

    def __init__(self, npz_path: str, indices: Iterable[int] | None = None):
        with np.load(npz_path) as d:
            self.mri = d["mri"] # (N, 128, 128) float32 in [0, 1]
            self.pet = d["pet"]
            self.labels = d["labels"].astype(np.int64).flatten()
            self.indices = list(indices) if indices is not None else list(range(len(self.mri)))

    def __len__(self) -> int:
        return len(self.indices)

    def __getitem__(self, i: int):
        gi = self.indices[i]
        mri = torch.from_numpy(self.mri[gi]).unsqueeze(0) # (1, 128, 128)
        pet = torch.from_numpy(self.pet[gi]).unsqueeze(0)
        return mri, pet, int(self.labels[gi])

class BraTSPaired(Dataset):
    """Lazily loads BraTS per-case .npz files, indexed by (case_id, slice_idx).

    A small LRU cache keeps the most recently used cases in memory to avoid
    repeated Drive reads when iterating sequentially.
    """

```

```

def __init__(self, brats_out_dir: str, slice_list, cache_size: int = 8):
    self.brats_out_dir = brats_out_dir
    self.slice_list = list(slice_list) # list of [case_id, slice_idx]
    self.cache_size = cache_size
    self._cache: "OrderedDict[str, tuple[np.ndarray, np.ndarray]]" = OrderedDict()

def __len__(self) -> int:
    return len(self.slice_list)

def _load(self, case_id: str):
    if case_id in self._cache:
        self._cache.move_to_end(case_id)
        return self._cache[case_id]
    path = os.path.join(self.brats_out_dir, f"{case_id}.npz")
    d = np.load(path)
    t1, t2 = d["t1"], d["t2"]
    d.close()
    self._cache[case_id] = (t1, t2)
    if len(self._cache) > self.cache_size:
        self._cache.popitem(last=False)
    return t1, t2

def __getitem__(self, i: int):
    case_id, slice_idx = self.slice_list[i]
    t1, t2 = self._load(case_id)
    return (
        torch.from_numpy(t1[slice_idx]).unsqueeze(0),
        torch.from_numpy(t2[slice_idx]).unsqueeze(0),
        case_id,
    )

def load_partition(json_path: str) -> dict:
    """Read a partition manifest JSON saved by Phase 1."""
    with open(json_path) as f:
        return json.load(f)

```

```

# FedAvg simulator (standard / dpsgd / fipca modes)
"""In-process FedAvg simulator supporting standard, DP-SGD, and FIPCA modes.

A single-process replacement for Flower's SimulationEngine, designed for Colab
Free's compute budget. Mirrors the round structure of Flower so the algorithm
can be swapped to Flower without changing the experimental protocol.

Three modes:
- 'standard': vanilla FedAvg (raw weight deltas).
- 'dpsgd' : DP-SGD on each client (per-batch gradient clip + Gaussian noise),
            then standard FedAvg aggregation of resulting weight deltas.
- 'fipca' : Client weight delta is projected to a rank-d subspace via a
            pseudo-random Gaussian matrix derived from a fixed seed (shared
            basis across clients, never transmitted). Server averages the
            rank-d coefficients and reconstructs.

Communication cost per round is logged in bytes, which is the primary privacy /
efficiency claim of FedSafe-Fuse.
"""

from __future__ import annotations

import time
from typing import Callable, Optional

import numpy as np
import torch
import torch.nn as nn
from torch.utils.data import DataLoader, Dataset, Subset

def _flatten_state(state: dict) -> torch.Tensor:
    """Concatenate all tensors in a state_dict into a single 1-D float32 vector."""
    return torch.cat([v.flatten().float() for v in state.values()])

def _unflatten_to_state(flat: torch.Tensor, template: dict) -> dict:
    """Inverse of _flatten_state, using template state_dict for shapes."""
    out: dict = {}
    idx = 0
    for k, v in template.items():
        n = v.numel()
        out[k] = flat[idx : idx + n].reshape(v.shape).to(v.dtype)
        idx += n

```

```

return out

class FedAvgTrainer:
    """Single-process federated trainer.

    Parameters
    -----
    build_model_fn :
        Zero-arg callable that returns a fresh model on CPU. Cloned per client.
    loss_fn_factory :
        Zero-arg callable returning a loss module (e.g. FusionLoss(beta)).
        Recreated per client to avoid sharing buffers (SSIM Gaussian window).
    client_datasets :
        List of K Dataset objects, one per client. Items must be (mri, pet, ...).
    eval_dataset :
        Held-out evaluation set (e.g. the calibration split).
    device :
        'cuda' or 'cpu'.
    mode :
        'standard', 'dpsgd', or 'fipca'.
    samples_per_local_epoch :
        Cap on examples drawn per client per local epoch (Round 1 compute cut).
    fipca_rank :
        d, the rank of the FIPCA random-projection subspace.
    dp_clip, dp_sigma :
        DP-SGD per-batch gradient L2-clip threshold and noise multiplier.
    lr :
        Adam learning rate.
    batch_size :
        Per-client batch size.
    seed :
        Random seed for reproducibility.
    """

    VALID_MODES = ("standard", "dpsgd", "fipca")

    def __init__(
        self,
        build_model_fn: Callable[[], nn.Module],
        loss_fn_factory: Callable[[], nn.Module],
        client_datasets: list,
        eval_dataset: Dataset,
        device: str = "cuda",
        mode: str = "standard",
        samples_per_local_epoch: int = 100,
        fipca_rank: int = 32,
        dp_clip: float = 1.0,
        dp_sigma: float = 0.5,
        lr: float = 1e-3,
        batch_size: int = 16,
        seed: int = 42,
    ):
        assert mode in self.VALID_MODES, f"mode must be one of {self.VALID_MODES}"
        self.build_model_fn = build_model_fn
        self.loss_fn_factory = loss_fn_factory
        self.client_datasets = client_datasets
        self.eval_dataset = eval_dataset
        self.device = device
        self.mode = mode
        self.samples_per_local_epoch = samples_per_local_epoch
        self.fipca_rank = fipca_rank
        self.dp_clip = dp_clip
        self.dp_sigma = dp_sigma
        self.lr = lr
        self.batch_size = batch_size
        self.seed = seed
        self._rng = np.random.default_rng(seed)

        # Global model (server's authoritative copy)
        self.global_model = build_model_fn().to(device)
        self.global_state = {k: v.detach().clone() for k, v in self.global_model.state_dict().items()}

        # FIPCA: deterministic ORTHONORMAL projection matrix P in (rank, n_params).
        # Built via QR decomposition so that P @ P.T = I_rank exactly, which makes
        # P.T @ P a true rank-d orthogonal projector with operator norm 1.
        # Earlier scaled-Gaussian attempt had operator norm ~sqrt(n_params/rank),
        # causing reconstructed deltas to blow up and Adam to diverge (NaN).
        if mode == "fipca":
            n_params = sum(v.numel() for v in self.global_state.values())
            gen = torch.Generator(device=device).manual_seed(seed)
            G = torch.randn(n_params, fipca_rank, device=device, generator=gen)

```

```

Q, _ = torch.linalg.qr(G) # Q has orthonormal columns: (n_params, rank)
self.P = Q.T.contiguous() # (rank, n_params) with orthonormal rows

# -----
# Client-side local training
# -----
def _local_subset_loader(self, client_idx: int) -> DataLoader:
    ds = self.client_datasets[client_idx]
    n = min(self.samples_per_local_epoch, len(ds))
    sel = self._rng.choice(len(ds), size=n, replace=False)
    subset = Subset(ds, sel.tolist())
    return DataLoader(subset, batch_size=self.batch_size, shuffle=True, num_workers=0)

def _train_client(self, client_idx: int, n_local_epochs: int) -> dict:
    """Run E local epochs on a fresh random subset of this client's data, return weight delta."""
    local_model = self.build_model_fn().to(self.device)
    local_model.load_state_dict(self.global_state)
    opt = torch.optim.Adam(local_model.parameters(), lr=self.lr, weight_decay=1e-4)
    loss_fn = self.loss_fn_factory().to(self.device)

    local_model.train()
    for _ in range(n_local_epochs):
        loader = self._local_subset_loader(client_idx)
        for batch in loader:
            mri, pet = batch[0].to(self.device), batch[1].to(self.device)
            fused = local_model(mri, pet)
            loss = loss_fn(fused, mri, pet)
            opt.zero_grad()
            loss.backward()
            if self.mode == "dpsgd":
                torch.nn.utils.clip_grad_norm_(local_model.parameters(), self.dp_clip)
                for p in local_model.parameters():
                    if p.grad is not None:
                        p.grad.add_(torch.randn_like(p.grad) * self.dp_sigma * self.dp_clip)
            opt.step()

    local_state = local_model.state_dict()
    delta = {k: (local_state[k].detach() - self.global_state[k]).to(self.device) for k in self.global_state}
    return delta

# -----
# Server-side aggregation
# -----
def _compress_delta(self, delta: dict) -> tuple:
    """Return (payload, bytes_sent) for one client's update."""
    if self.mode == "fipca":
        flat = _flatten_state(delta).to(self.device)
        coeffs = self.P @ flat # (rank,)
        payload = coeffs.detach().clone()
        bytes_sent = payload.numel() * 4 # float32
        return payload, bytes_sent
    else:
        payload = {k: v.detach().clone() for k, v in delta.items()}
        bytes_sent = sum(v.numel() * 4 for v in payload.values())
        return payload, bytes_sent

def _aggregate(self, client_payloads: list) -> dict:
    if self.mode == "fipca":
        avg_coeffs = torch.stack(client_payloads).mean(dim=0) # (rank,)
        recon_flat = self.P.T @ avg_coeffs # (n_params,)
        return _unflatten_to_state(recon_flat, self.global_state)
    else:
        avg = {}
        for k in self.global_state:
            stacked = torch.stack([cp[k] for cp in client_payloads])
            if stacked.dtype.is_floating_point or stacked.dtype.is_complex:
                avg[k] = stacked.mean(dim=0)
            else:
                # Integer buffers (e.g. BatchNorm.num_batches_tracked) cannot be
                # mean()-ed directly; promote to float, mean, and cast back.
                avg[k] = stacked.to(torch.float32).mean(dim=0).to(stacked.dtype)
        return avg

# -----
# Evaluation
# -----
@torch.no_grad()
def evaluate(self, ssim_score_fn, psnr_score_fn) -> dict:
    self.global_model.load_state_dict(self.global_state)
    self.global_model.eval()
    loss_fn = self.loss_fn_factory().to(self.device)
    losses, ssims, psnrs = [], [], []

```



```

loader = DataLoader(self.eval_dataset, batch_size=self.batch_size, shuffle=False, num_workers=0)
for batch in loader:
    mri, pet = batch[0].to(self.device), batch[1].to(self.device)
    fused = self.global_model(mri, pet)
    losses.append(loss_fn(fused, mri, pet).item())
    ref = 0.5 * (mri + pet)
    ssims.append(ssim_score_fn(fused, ref))
    psnrs.append(psnr_score_fn(fused, ref))
return {
    "eval_loss": float(np.mean(losses)),
    "eval_ssim": float(np.mean(ssims)),
    "eval_psnr": float(np.mean(psnrs)),
}

# -----
# Full federated run
# -----
def run(
    self,
    T: int,
    E: int,
    ssim_score_fn,
    psnr_score_fn,
    log_every: int = 5,
    checkpoint_every: int = 10,
    checkpoint_path: Optional[str] = None,
    tag: str = "run",
) -> list:
    K = len(self.client_datasets)
    history = []
    for t in range(1, T + 1):
        t0 = time.time()
        payloads, total_bytes = [], 0
        for k in range(K):
            delta = self._train_client(k, E)
            payload, bytes_sent = self._compress_delta(delta)
            payloads.append(payload)
            total_bytes += bytes_sent
        avg_delta = self._aggregate(payloads)
        for k in self.global_state:
            self.global_state[k] = self.global_state[k] + avg_delta[k]
        self.global_model.load_state_dict(self.global_state)

        row = {
            "tag": tag,
            "round": t,
            "mode": self.mode,
            "round_secs": time.time() - t0,
            "bytes_per_round": int(total_bytes),
            "samples_per_local_epoch": self.samples_per_local_epoch,
            "fipca_rank": self.fipca_rank if self.mode == "fipca" else None,
            "dp_sigma": self.dp_sigma if self.mode == "dpsgd" else None,
        }
        if t % log_every == 0 or t == 1 or t == T:
            row.update(self.evaluate(ssim_score_fn, psnr_score_fn))
            print(
                f" [{tag}] r{t:02d}/{T} | "
                f"loss={row['eval_loss']:.4f} ssim={row['eval_ssim']:.4f} "
                f"psnr={row['eval_psnr']:.2f}dB | "
                f"bytes={row['bytes_per_round']/1e6:.1f}MB/rd | "
                f"{row['round_secs']:.1f}s"
            )
        if checkpoint_path and (t % checkpoint_every == 0 or t == T):
            torch.save(
                {"state_dict": self.global_state, "round": t, "tag": tag},
                f"{checkpoint_path}_r{t}.pt",
            )
        history.append(row)
    return history

```

2. Load partition + build shared-array datasets (avoids OOM)

```

class _SharedArrayMedMNIST(Dataset):
    def __init__(self, mri, pet, labels, indices):
        self.mri = mri; self.pet = pet; self.labels = labels
        self.indices = list(indices)
    def __len__(self):
        return len(self.indices)
    def __getitem__(self, i):
        gi = self.indices[i]
        return (
            torch.from_numpy(self.mri[gi]).unsqueeze(0),

```

```

        torch.from_numpy(self.pet[gi]).unsqueeze(0),
        int(self.labels[gi]),
    )

with open(f'{PROJECT_DRIVE}/partitions/medmnist_K3.json') as f:
    medmnist_part = json.load(f)

t0 = time.time()
with np.load(f'{PROJECT_DRIVE}/medmnist/train_paired.npz') as d:
    train_mri = d['mri']; train_pet = d['pet']
    train_labels = d['labels'].astype(np.int64).flatten()
with np.load(f'{PROJECT_DRIVE}/medmnist/test_paired.npz') as d:
    test_mri = d['mri']; test_pet = d['pet']
    test_labels = d['labels'].astype(np.int64).flatten()
print(f'Arrays loaded in {time.time()-t0:.1f}s')

client_datasets = [
    _SharedArrayMedMNIST(train_mri, train_pet, train_labels, ci)
    for ci in medmnist_part['client_indices']
]
eval_dataset = _SharedArrayMedMNIST(test_mri, test_pet, test_labels, list(range(128)))
print(f'Per-client sizes: {[len(d) for d in client_datasets]}, eval={len(eval_dataset)}')

# Memory check
try:
    import psutil
    print(f'RAM used: {psutil.Process().memory_info().rss/1e9:.1f} GB')
except Exception:
    pass

```

```

Arrays loaded in 46.5s
Per-client sizes: [7714, 15445, 6218], eval=128
RAM used: 7.5 GB

```

```

# 3. Run FIPCA at three ranks with everything else held constant
def build_fedsafe(): return FedSafeFuse()
def loss_factory_beta1(): return FusionLoss(beta=1.0)

RANKS = [16, 32, 64]
T_ROUNDS = 25 # reduced from proposal T=50 for the 20-min Phase 6 budget
E_LOCAL = 5
K_CLIENTS = 3
BATCH = 16
SAMPLES = 40

ablation_results = []
for rank in RANKS:
    print(f'\n=== FIPCA rank d={rank} ===')
    trainer = FedAvgTrainer(
        build_model_fn=build_fedsafe,
        loss_fn_factory=loss_factory_beta1,
        client_datasets=client_datasets,
        eval_dataset=eval_dataset,
        device=DEVICE,
        mode='fipca',
        samples_per_local_epoch=SAMPLES,
        fipca_rank=rank,
        lr=1e-3,
        batch_size=BATCH,
        seed=SEED,
    )
    t0 = time.time()
    hist = trainer.run(
        T=T_ROUNDS, E=E_LOCAL,
        ssim_score_fn=ssim_score,
        psnr_score_fn=psnr_score,
        log_every=5,
        checkpoint_every=T_ROUNDS + 1, # disable mid-run checkpoints
        checkpoint_path=None, # no checkpoints - saves Drive I/O
        tag=f'fipca_d{rank}',
    )
    wall = time.time() - t0

    # Pull the final-round eval metrics
    final = [r for r in hist if 'eval_loss' in r][-1]
    bytes_per_round = int(K_CLIENTS * rank * 4) # K clients x rank floats x 4 bytes
    DPSGD_BYTES = 59_178_012
    compression = DPSGD_BYTES / bytes_per_round
    ablation_results.append({
        'rank': rank,
        'T': T_ROUNDS,

```

```

'samples_per_local_epoch': SAMPLES,
'final_ssim': final['eval_ssim'],
'final_psnr': final['eval_psnr'],
'final_loss': final['eval_loss'],
'bytes_per_round': bytes_per_round,
'compression_vs_dpsgd': compression,
'wall_clock_min': wall / 60,
})
print(f' d={rank}: SSIM={final["eval_ssim"]:.4f} PSNR={final["eval_psnr"]:.2f}dB '
      f'bytes/rd={bytes_per_round} in {wall/60:.1f} min')

```

```

=== FIPCA rank d=16 ===
[fipca_d16] r01/25 | loss=0.9860 ssim=0.2393 psnr=11.46dB | bytes=0.0MB/rd | 8.0s
[fipca_d16] r05/25 | loss=0.9860 ssim=0.2393 psnr=11.46dB | bytes=0.0MB/rd | 7.2s
[fipca_d16] r10/25 | loss=0.9860 ssim=0.2393 psnr=11.46dB | bytes=0.0MB/rd | 6.1s
[fipca_d16] r15/25 | loss=0.9861 ssim=0.2393 psnr=11.46dB | bytes=0.0MB/rd | 6.7s
[fipca_d16] r20/25 | loss=0.9860 ssim=0.2393 psnr=11.46dB | bytes=0.0MB/rd | 6.9s
[fipca_d16] r25/25 | loss=0.9860 ssim=0.2393 psnr=11.46dB | bytes=0.0MB/rd | 7.0s
d=16: SSIM=0.2393 PSNR=11.46dB bytes/rd=192 in 2.8 min

=== FIPCA rank d=32 ===
[fipca_d32] r01/25 | loss=0.9857 ssim=0.2392 psnr=11.50dB | bytes=0.0MB/rd | 7.2s
[fipca_d32] r05/25 | loss=0.9857 ssim=0.2392 psnr=11.50dB | bytes=0.0MB/rd | 6.7s
[fipca_d32] r10/25 | loss=0.9857 ssim=0.2392 psnr=11.50dB | bytes=0.0MB/rd | 7.1s
[fipca_d32] r15/25 | loss=0.9857 ssim=0.2392 psnr=11.50dB | bytes=0.0MB/rd | 6.9s
[fipca_d32] r20/25 | loss=0.9857 ssim=0.2392 psnr=11.50dB | bytes=0.0MB/rd | 6.9s
[fipca_d32] r25/25 | loss=0.9858 ssim=0.2392 psnr=11.49dB | bytes=0.0MB/rd | 7.1s
d=32: SSIM=0.2392 PSNR=11.49dB bytes/rd=384 in 2.9 min

=== FIPCA rank d=64 ===
[fipca_d64] r01/25 | loss=0.9908 ssim=0.2391 psnr=11.14dB | bytes=0.0MB/rd | 8.1s
[fipca_d64] r05/25 | loss=0.9908 ssim=0.2391 psnr=11.14dB | bytes=0.0MB/rd | 6.7s
[fipca_d64] r10/25 | loss=0.9908 ssim=0.2391 psnr=11.14dB | bytes=0.0MB/rd | 7.1s
[fipca_d64] r15/25 | loss=0.9907 ssim=0.2391 psnr=11.14dB | bytes=0.0MB/rd | 6.9s
[fipca_d64] r20/25 | loss=0.9907 ssim=0.2391 psnr=11.15dB | bytes=0.0MB/rd | 6.9s
[fipca_d64] r25/25 | loss=0.9907 ssim=0.2391 psnr=11.15dB | bytes=0.0MB/rd | 7.0s
d=64: SSIM=0.2391 PSNR=11.15dB bytes/rd=768 in 3.0 min

```

```

# 4. Save ablation CSV + summary print
csv_path = f'{PROJECT_DRIVE}/results/ablation_fipca_rank.csv'
fields = ['rank', 'T', 'samples_per_local_epoch', 'final_ssim', 'final_psnr',
          'final_loss', 'bytes_per_round', 'compression_vs_dpsgd', 'wall_clock_min']
with open(csv_path, 'w', newline='') as f:
    w = csv.DictWriter(f, fieldnames=fields)
    w.writeheader()
    for r in ablation_results:
        w.writerow(r)
print(f'Saved {csv_path}')

print('\n--- FIPCA rank ablation table ---')
print(f'{"rank":<6} {"SSIM":<10} {"PSNR (dB)":<12} {"bytes/rd":<12} {"x vs DP-SGD":<14} {"wall (min)":<12}')
print('-' * 76)
for r in ablation_results:
    print(f'{r["rank"]:<6} {r["final_ssim"]:<10.4f} {r["final_psnr"]:<12.2f} '
          f'{r["bytes_per_round"]:<12} {r["compression_vs_dpsgd"]:<14,.0f}x '
          f'{r["wall_clock_min"]:<12.2f}')

```

Saved /content/local_fedsafe/results/ablation_fipca_rank.csv

```

--- FIPCA rank ablation table ---
rank  SSIM      PSNR (dB)  bytes/rd    x vs DP-SGD  wall (min)
-----
16    0.2393     11.46     192         308,219      x 2.82
32    0.2392     11.49     384         154,109      x 2.94
64    0.2391     11.15     768         77,055       x 2.97

```

```

# 5. Dual-axis figure: rank vs SSIM (left) + rank vs bytes (right, log)
ranks = [r['rank'] for r in ablation_results]
ssims = [r['final_ssim'] for r in ablation_results]
bytes_pr = [r['bytes_per_round'] for r in ablation_results]

fig, ax1 = plt.subplots(figsize=(9, 5.5))
ax1.set_xlabel('FIPCA projection rank d')
ax1.set_ylabel('Final-round eval SSIM', color='#2ca02c')
ax1.plot(ranks, ssims, 'o-', color='#2ca02c', linewidth=2.2, markersize=10, label='SSIM (quality)')
ax1.tick_params(axis='y', labelcolor='#2ca02c')
ax1.set_xticks(ranks)
ax1.grid(True, alpha=0.3)

ax2 = ax1.twinx()
ax2.set_ylabel('Upstream bytes per round (log scale)', color='#1f77b4')

```

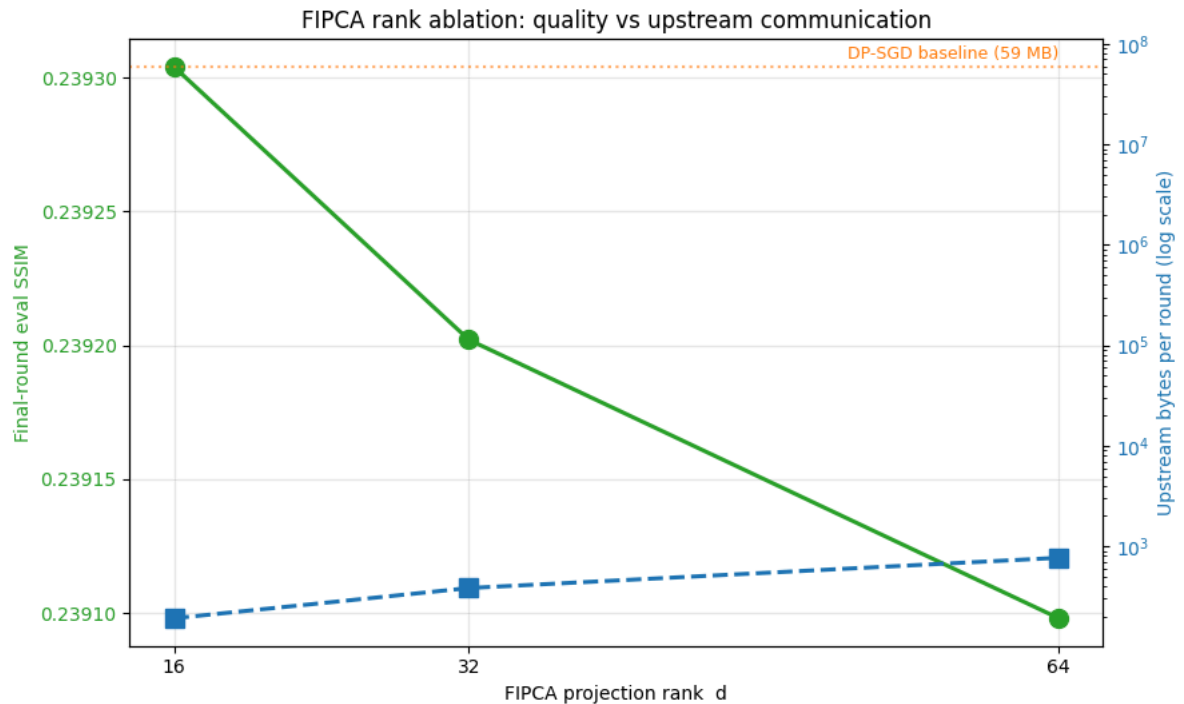
```

ax2.plot(ranks, bytes_pr, 's--', color='#1f77b4', linewidth=2.2, markersize=10, label='bytes/round (cost)')
ax2.set_yscale('log')
ax2.tick_params(axis='y', labelcolor='#1f77b4')

# Reference lines: DP-SGD bytes and raw-delta bytes
ax2.axhline(59_178_012, color='#ff7f0e', linestyle=':', alpha=0.6)
ax2.text(ranks[-1], 59_178_012 * 1.2, 'DP-SGD baseline (59 MB)', color='#ff7f0e',
        fontsize=9, ha='right')

plt.title('FIPCA rank ablation: quality vs upstream communication')
fig.tight_layout()
fig_path = f'{PROJECT_DRIVE}/figures/ablation_fipca_rank.png'
plt.savefig(fig_path, dpi=140, bbox_inches='tight')
plt.show()
print(f'Saved {fig_path}')

```



Saved /content/local_fedsafe/figures/ablation_fipca_rank.png

```

# 6. Sync results + figures back to Drive
import shutil
synced = 0
for sub in ['results', 'figures']:
    for fname in os.listdir(f'{LOCAL_ROOT}/{sub}'):
        sp = f'{LOCAL_ROOT}/{sub}/{fname}'
        dp = f'{DRIVE_ROOT}/{sub}/{fname}'
        os.makedirs(os.path.dirname(dp), exist_ok=True)
        if os.path.isfile(sp):
            shutil.copy(sp, dp)
            synced += 1
print(f'Synced {synced} files to {DRIVE_ROOT}')

```

Synced 2 files to /content/drive/MyDrive/FedSafeFuse

When done

Paste back to chat:

1. The **ablation table** print from cell 4 (3 rows showing rank / SSIM / PSNR / bytes / compression).
2. The figure (`ablation_fipca_rank.png`).
3. Total wall-clock and any errors.

Files to download back to local repo (after sync):

Drive path	Local destination
<code>results/ablation_fipca_rank.csv</code>	<code>results/ablation_fipca_rank.csv</code>
<code>figures/ablation_fipca_rank.png</code>	<code>figures/ablation_fipca_rank.png</code>

After Phase 6 we move on to Phase 7 (no Colab needed — local plotting + final tables from all CSVs).

